

02-lab-work-speaking-script

Speaking Script: S3 Code Review and Lab Work

Internal notes. Walks the 23-sheet presentation PDF, sheet by sheet: the one thing on each, the number to say, and the line that carries it.

You present	group 5 - s3 code review & lab work.pdf , 23 sheets
Backup document	detailed non-functional-requirements.pdf , 8 sheets
Target	12 to 14 minutes, then questions

Have open: the PDF, a terminal at the repo root, GitHub with green checks, the live site already warmed.

Two rules. Never claim quality, show what is enforced. And **run the tools live** on sheets 12 and 13, because he said the codebase does not need walking.

The six numbers to know cold

If you remember nothing else, these are the session.

Number	What it is
Grade A, all 43 files	Maintainability index. No exceptions
3.90 average, grade A	Cyclomatic complexity over 345 blocks
0 blocks at E or F	Was 1. We fixed it during the review
310 tests, 65% coverage	Run against a real Postgres in CI
89.6% type precision	mypy strict, 0 errors
20,263 evidence units	Real reviews, 3 sources, 143 variants

Sheets 1 to 2: open (40 seconds)

Revix is an evidence-traceable review system for Indian cars and two-wheelers. A star average hides who rated and whether they agreed, so **we rebuild the verdict from the reviews underneath and keep every number traceable to the sentences it came from.**

Twenty thousand review units, three sources, 143 variants. Real data.

On contents: "Five sections. Most of the time on section three, the measured code quality, since that is what today is for."

Do not demo the product. That is the 16 October session.

Sheets 3 to 6: codebase (2.5 minutes)

Sheet	Say
3 Shape	Three Python packages and a Next.js app, ~ 17,600 lines . API is deliberately the smallest. 134 of 143 variants publish a verdict; nine are held back below the evidence floor
4 Structure	The one rule: dependencies run one direction. Core depends on neither other package. Enforced by import linting in CI, not by us remembering
5 Pipeline	Five stages, each runnable alone. Point at one property: fusion writes the citations before any prose exists , so a citation cannot be wrong
6 What it serves	15 endpoints, 11 routes, 19 test modules. CI also checks migrations apply, match the models, and reverse cleanly

The line to land on sheet 4:

In a single package this would be a naming convention, and naming conventions lose. Here **the API physically cannot import the pipeline.**

Sheets 7 to 10: frameworks (2.5 minutes)

Sheet	Say
7 The table	Let him scan it. "Every choice is a written decision record, with the alternative we rejected. Nine of them"
8 Two ADRs	The two below. Slow down here
9 Where it runs	Four platforms. The write path runs overnight on the left, the read path along the bottom per request
10 Toolchain	Ten tools, and the column that matters is "fails the build"

ADR 0002, orchestration:

Scheduled GitHub Actions instead of Airflow. One nightly batch, no fan-out, no backfill. Actions already holds our secrets and schedules. **Choosing Airflow would have been choosing a tool because it looks professional rather than because the problem needs it.**

ADR 0004, the honest result:

We built an aspect classifier. **It lost to the lexicon it learned from by 0.29 macro F1, so it ships disabled**, and the number is published on our metrics page.

The alternative was shipping a model because we had built one.

Do not apologise for that. A negative result you published beats a positive one you did not check.

On sheet 10, the check to defend hardest:

CI regenerates the TypeScript client from the live schema and **fails if the committed types differ**. It is structural: it does not depend on anybody remembering.

Sheets 11 to 19: code quality (5 minutes, the core)

Sheet 11, the scoreboard

Six tools. Radon scores it, Xenon enforces it, Ruff and mypy cover style and types, PyTest runs the suite. **Every file grade A, average complexity A.**

Sheets 12 and 13, run these live

Type them. Do not read the slide.

```
uv run radon mi packages/revix_core/src pipeline/src apps/api/src -s
```

Maintainability index: how hard a file will be to change in six months. Above twenty is grade A. **All forty-three of our files are A.**

```
uv run radon cc packages/revix_core/src pipeline/src apps/api/src -a -s -nC
```

Complexity counts the paths through a block, which is also the minimum number of tests to cover it. **Average 3.90 across 345 blocks. 91.8% of the codebase is A or B. Nothing at E or F.**

Sheet 14, what the measurement changed

The strongest item in the session.

One function graded **E at 33**, the only E and forty percent worse than the next. It was doing two jobs: drawing samples, and summarising them. Neither half is complicated. Doing both in one place was.

Split into five functions, and **the codebase now has no E at all**. The thirteen tests over it passed unchanged, which is the only reason that refactor was safe.

Then volunteer the two at D, before he finds them:

`fuse_variant` is the scoring engine, `run_connector` is the ingest loop holding three failure modes open so one dead source never stops the others. **In both, the branching is the requirement, not a style accident.** And a refactor that changes a verdict is worse than a D grade.

Sheets 15 to 17, one tool each

Sheet	Number	Line
15 Xenon, Ruff	gate passes, 0 findings	"Radon reports; Xenon refuses . A change that makes any grade worse does not merge"
16 mypy	0 errors, 89.6% precise	"The 10.44% imprecise is at scikit-learn and JSON edges, not in our logic"
17 PyTest	310 tests, 65%	"266 need no database. Coverage is deliberately not gated , and the next sheet says why"

Sheet 18, the tool we skipped

Mutation testing was the most interesting on your list. **Mutmut has no native Windows support, and all three of us are on Windows.** A tool only one machine can run is one that stops being run.
The question it answers, whether the tests would catch a bug, we answer with evidence.

Then pick two defects, not all five:

Our macro-F1 averaged over zero-support aspects, so a perfect system scored 0.22. **Our evaluation was flattering us**, and a test caught it.
And a connector recorded our own search term as the source's answer, which means **we would have been citing ourselves as evidence.**

Sheet 19, the generated reports

Two of the six produce a report you can open. **coverage.py** writes a per-line browsable report, **mypy** writes the per-module precision report the 89.6% comes from. Ruff and Xenon deliberately produce only an exit code, because a gate's answer is pass or fail.

Sheets 20 to 22: non-functional requirements (3 minutes)

Sheet 20, lead with the architecture

All nine audited against the code rather than the proposal. **Six were already satisfied by one decision**, and I would rather explain it than read a list.

The write path and the read path are strictly separated. **Every endpoint is one indexed read of a row that already exists. No model runs during a request.**

That gives four at once: performance, because nothing on the read path grows with the corpus; scalability, because ten times the data changes the night and not the request; reliability, because a page cannot fail when a third-party model is down; availability, because a demo depends on nothing remote.

Pause. Then: **22 to 35 ms warm, 96 to 247 from India, against a 300 ms target.**

Sheet 21, the nine

Eight met, one stated honestly. The audit found **five real gaps and closed all five**: no security headers anywhere, unbounded database waits, no rate limiting, no compression, no request identity.

Sheet 22, the finding to lead with

If one NFR point lands, make it this.

Our health endpoint is deliberately written to report an unreachable database as a 503 rather than a stack trace. Written that way on purpose. There is a test for it.

It could not actually do it. No connect timeout, so the driver sat on the OS TCP timeout while the platform's probe gave up first. **A correct 503 became no answer at all**, and a hang is worse than an error because nothing downstream can react.

The lesson is the difference between writing a non-functional requirement and verifying one. We had the code, the intent and the test, and it still was not met.

Close with the limits, before he finds them:

The rate limiter is per instance and in memory, not a global quota. And one free-tier instance is resilient, not highly available. Both are the first things that would need replacing.

Sheet 23: close (30 seconds)

Every figure is reproducible in under a minute and the commands are on this sheet. The detailed non-functional requirements document has the full working if you want it. The site is live now, and the working demo is the next submission.

Questions, short answers

"Why is the classifier disabled?"

It lost to the lexicon it was trained from by 0.29 macro F1. Shipping it anyway would contradict the argument of the project. Published, not dropped.

"How do you know the tests are worth anything?"

By what they caught. *Give two defects from sheet 18.* And that is why we do not gate coverage: a threshold is trivially met by tests that assert nothing.

"Why not Airflow?"

One nightly batch, no fan-out, no backfill. ADR 0002 records it and the trigger for revisiting.

"Have you actually measured the 300 ms?"

Every response carries `X-Response-Time-ms`. 22 to 35 warm, 96 to 247 from India. It holds because of the architecture, not tuning.

"Only three sources?"

Three that permit us. Team-BHP returns 403 and two others disallow us in robots.txt. **We respect that rather than spoof a browser.**

"Is the data real?"

All of it. Twenty thousand review units. Nine variants sit below the evidence floor and are held back rather than shown.

"Can I see the code?"

Which part? Default to `admin.py` for the fail-closed authentication, or `fuse.py` for the D-grade scoring engine from sheet 14.

If something breaks

Problem	Do
A command errors live	Read it out and diagnose. Calm diagnosis beats a flustered excuse
CI red on the day	Open the job and read it. A red build you can explain beats a green one you cannot
Site is cold	"That is the free-tier cold start." Turn it into the availability point
Asked for something unbuilt	"Not built, here is what we chose instead and why." Never invent
Lost	Back to the five headings: codebase, frameworks, code quality, NFRs, run it live

Never say "it usually works." Show it, or say plainly it is broken and what you would check first.